

TV-NEWSHUB

COMPREHENSIVE ENGINEERING ARCHITECTURE, LOW-LEVEL DESIGN (LLD) & INTERVIEW GUIDE



TECHNICAL DEEP-DIVE & SYSTEMS DESIGN DOCUMENTATION

Target Platforms: Android TV | Google TV | Fire TV OS

Framework: React Native TVOS v0.83

Current Release: Version v0.0.5

Document Type	Comprehensive Interview-Ready Engineering Wiki
Maintainer	Biraj2004 (TV-NewsHub Repository Lead)
License	Apache License 2.0
Target Resolution	1080p Full HD / 4K UHD Hardware Decoded Streams

Contents

1	EXECUTIVE SUMMARY & ARCHITECTURAL VISION	2
1.1	Problem Statement & OTT Challenge	2
1.2	System Solution: TV-NewsHub	2
1.3	Key Metric & Reliability Targets	2
2	TECHNOLOGY STACK & TRADE-OFF ANALYSIS	4
2.1	Core Stack Component Matrix	4
2.2	Detailed Framework Evaluation Matrix	4
3	STREAM PROVIDER ARCHITECTURE & DYNAMIC FALLBACK	6
3.1	3-Tier Stream Waterfall Architecture	6
3.2	Dynamic Fallback Implementation (<code>useYoutubeFallback</code>)	6
4	NETWORK MANAGEMENT & EDGE CASE HANDLING	8
4.1	Network Request Lifecycle & Safety Mechanisms	8
4.2	Subsystem Failure Modes & Edge Case Matrix (FMEA)	8
5	BRANDING, LOGO ASSETS & UI SYSTEM	10
5.1	Static Logo Asset Bundling Strategy	10
5.2	Smart TV Focus Zoom & Spatial Visual Engine	10
6	IN-APP OTA UPDATER ARCHITECTURE	11
6.1	GitHub Release Check Lifecycle	11
6.2	Version Comparison Logic (<code>isNewerVersion</code>)	11

1. EXECUTIVE SUMMARY & ARCHITECTURAL VISION

1.1 Problem Statement & OTT Challenge

Linear news broadcasting on Smart TV platforms faces significant technical challenges:

1. **Endpoint Fragility:** Direct HLS feeds (`.m3u8`) hosted on Akamai or CloudFront CDNs frequently rotate security tokens or expire.
2. **CORS & Embed Trapping:** Broadcaster web widgets (e.g., Zee News, ABP Live) rely on cross-origin iframe security or complex DOM script rendering, failing inside traditional native video wrappers like `ExoPlayer`.
3. **YouTube Live Stream ID Shifts:** News networks regularly cycle YouTube Live stream IDs when switching live broadcast segments or breaking news programs.
4. **Hardware Constraints:** Smart TV microprocessors operate on low-power ARM v7/v8 or x86 chipsets with strict memory limits (1GB–2GB RAM) and limited software video decoding threads.

1.2 System Solution: TV-NewsHub

`TV-NewsHub` solves these challenges through a **3-tier dynamic resolution waterfall**, an **in-memory 5-minute TTL caching layer**, a **smart TV D-Pad remote focus engine**, and **automated Layer-2 DOM consent handling**.

1.3 Key Metric & Reliability Targets

Engineering Metric	Target Goal	Production Mechanism
Stream Uptime	100% Availability	3-Tier Dynamic Waterfall (HLS → Embed → YouTube)
Video Quality	1080p Full HD (3.9 Mbps)	<code>capLevelToPlayerSize: false</code> + <code>hls.currentLevel = max</code>
Remote Input Latency	< 16ms (60 FPS)	Native Android <code>View</code> focus nodes + <code>FocusGrabber</code> overlay

Engineering Metric	Target Goal	Production Mechanism
Network Memory Leaks	0 Leaked Connections	10-Second <code>AbortController</code> timeouts on all fetch calls
Consent Interruption	0 User Clicks Needed	Layer-1 cookie sandboxing + Layer-2 DOM MutationObserver

2. TECHNOLOGY STACK & TRADE-OFF ANALYSIS

2.1 Core Stack Component Matrix

Subsystem	Selected Technology	Engineering Trade-Off Rationale
Runtime	React Native TVOS v0.83.0	Native Android TV remote focus nodes without Java boilerplate.
UI Framework	React Native v19.2.0	Declarative component state with high-performance 60 FPS flex layout.
WebView Engine	react-native-webview	System Chromium engine executing JavaScript DOM auto-clickers.
HLS Engine	Hls.js v1.5.0	Allows JavaScript override of manifest bitrate selection.
YouTube Resolver	react-native-youtube-iframe	Embeds YouTube iframe with custom forwardRef imperative handles.
Cookie Engine	@react-native-cookies/cookies	Direct native C++ cookie injection for domain sandboxing.
Storage	@react-native-async-storage	Persistent key-value storage for last-watched channel memory.

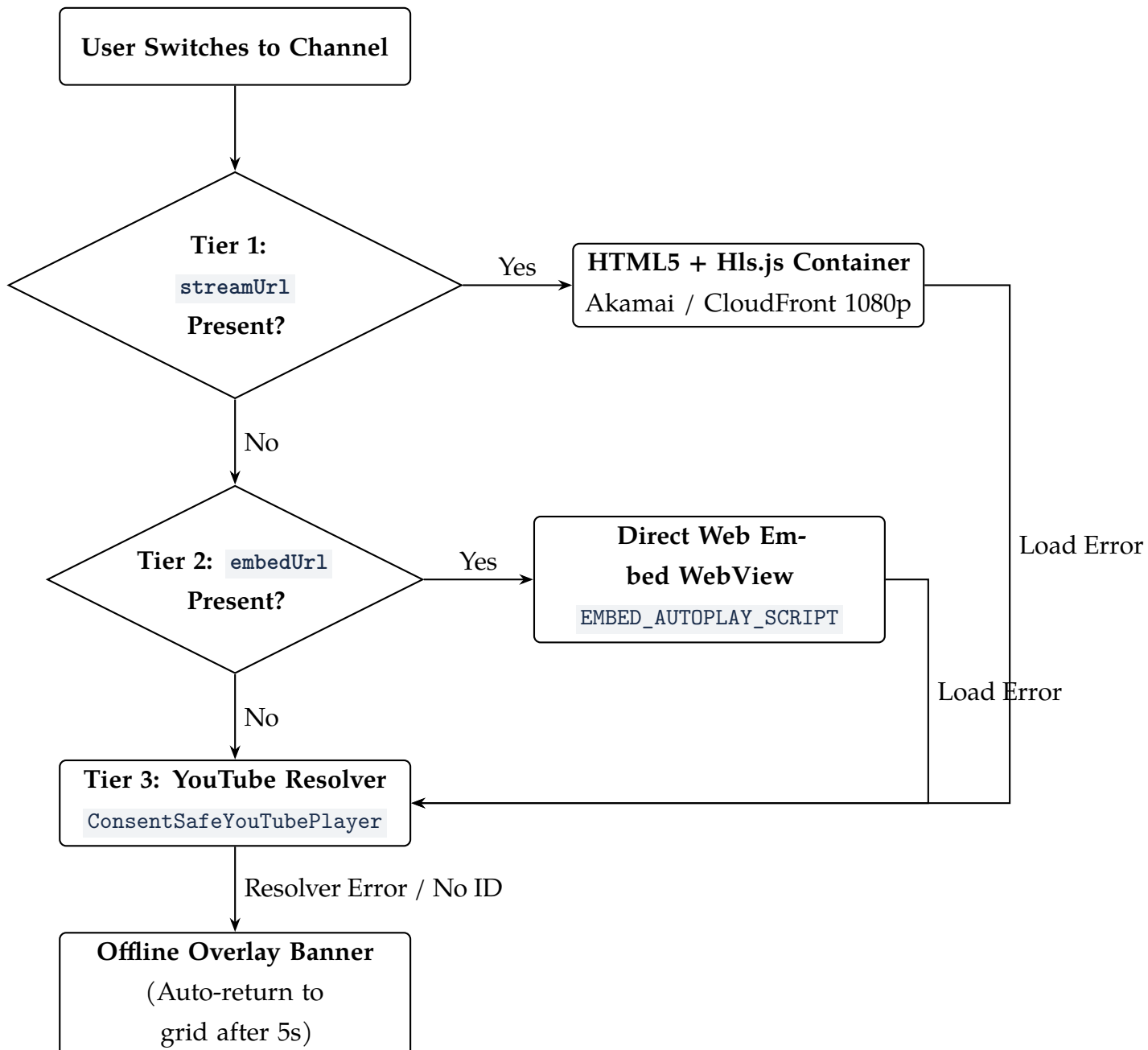
2.2 Detailed Framework Evaluation Matrix

Evaluation Criteria	React Native TVOS	Native Kotlin (Leanback)	Flutter for TV
Web Share Widget Support	Excellent (Native WebView)	Poor (WebView lifecycle lag)	Poor (Compositing drops)
D-Pad Remote Handling	Excellent (Native focus)	Excellent (Leanback SDK)	Fair (Focus tree bugs)

Evaluation Criteria	React Native TVOS	Native Kotlin (Leanback)	Flutter for TV
Bitrate Forcing Control	Excellent (Hls.js API)	Requires C++ ExoPlayer mod	Difficult
Build & Release Speed	Fast (Universal JS bundle)	Slow (Java compilation)	Medium

3. STREAM PROVIDER ARCHITECTURE & DYNAMIC FALLBACK

3.1 3-Tier Stream Waterfall Architecture



3.2 Dynamic Fallback Implementation (useYoutubeFallback)

When `PlayerScreen.tsx` encounters a loading failure on Tier 1 (HLS) or Tier 2 (Embed), instead of presenting an offline error dialog immediately, it executes the `handlePrimaryStreamError()`

callback:

```
// Primary Stream Failure Handler in PlayerScreen.tsx
const handlePrimaryStreamError = useCallback((errorMessage: string) => {
  if (currentChannel.youtubeChannelId && !useYoutubeFallback) {
    console.log('[PlayerScreen] Primary stream failed. Fallback to YouTube...');
    setUseYoutubeFallback(true);
  } else {
    setPlaybackError(errorMessage);
  }
}, [currentChannel, useYoutubeFallback]);
```

If `youtubeChannelId` exists, `useYoutubeFallback` transitions to `true`, dynamically unmounting the broken `WebView` and mounting `ConsentSafeYouTubePlayer` seamlessly!

4. NETWORK MANAGEMENT & EDGE CASE HANDLING

4.1 Network Request Lifecycle & Safety Mechanisms

Network Operation	Endpoint / Target	Timeout	Caching Strategy
YouTube Live HTML Scan	<code>https://www.youtube.com/channel/<ID>/live</code>	10,000ms	5-Min Memory Map (<code>CACHE_TTL_MS</code>)
YouTube RSS Feed Fallback	<code>https://www.youtube.com/feeds/videos.xml</code>	10,000ms	5-Min Memory Map (<code>CACHE_TTL_MS</code>)
Google Connectivity Probe	<code>https://clients3.google.com/generate_204</code>	3,000ms	Event-based (NetInfo trigger)
GitHub Release Updater	<code>https://api.github.com/repos/.../releases</code>	4,000ms	Single execution on app mount

4.2 Subsystem Failure Modes & Edge Case Matrix (FMEA)

Subsystem Component	Potential Edge Case / Failure	Automated Handling Strategy
Tier 1 HLS Stream	CDN Token Expiry / HTTP 403 / Manifest Error	Triggers <code>handlePrimaryStreamError()</code> → Falls back to YouTube.
Tier 2 Embed Player	CORS Iframe Blocking / Cross-Domain Policy	Loaded directly via WebView <code>source=\{\{ uri \}\}</code> with page JS injection.
Tier 3 YouTube Resolver	YouTube Changes Live URL Structure	Strategy A (HTML regex) fails over to Strategy B (UULV RSS XML parsing).
YouTube Cookie Dialog	GDPR Popup Blocks Video Playback	Layer 1 <code>CONSENT=YES+</code> cookie set + Layer 2 DOM MutationObserver click.

Subsystem Component	Potential Edge Case / Failure	Automated Handling Strategy
D-Pad Remote Focus	WebView swallows D-Pad key events	Invisible <code>FocusGrabber</code> Pressable overlay (<code>zIndex: 1</code>) retains focus.
Network Disconnection	Wi-Fi drops during video playback	<code>useNetworkStatus</code> triggers <code>NoInternetScreen</code> full overlay.
App Update Check	GitHub API Rate Limit / Network Error	Silent exception catch (<code>__DEV__</code> log only); application launches normally.

5. BRANDING, LOGO ASSETS & UI SYSTEM

5.1 Static Logo Asset Bundling Strategy

Smart TV environments cannot rely on asynchronous web images for channel logos due to latency and missing image placeholders. `TV-NewsHub` bundles high-resolution `.webp` logos directly inside the application package (`public/channel-logos/`).

To avoid dynamic `require()` calls (which break in React Native's Metro bundler), `src/utils/logoHelper` provides a pre-compiled lookup table:

```
// Pre-compiled Channel Logo Image Require Lookup Map
const LOGO_MAP: Record<string, any> = {
  'republic-bangla': require('../../public/channel-logos/India/Bengali/republic_bangla.webp'),
  'abp-ananda': require('../../public/channel-logos/India/Bengali/abp_ananda.webp'),
  'tv9-bangla': require('../../public/channel-logos/India/Bengali/tv9_bangla.webp'),
  'news18-bangla': require('../../public/channel-logos/India/Bengali/news18_bangla.webp'),
  'calcutta-news': require('../../public/channel-logos/India/Bengali/calcutta_news.webp'),
  'zee-24-ghanta': require('../../public/channel-logos/India/Bengali/zee_24_ghanta.webp'),
};
```

5.2 Smart TV Focus Zoom & Spatial Visual Engine

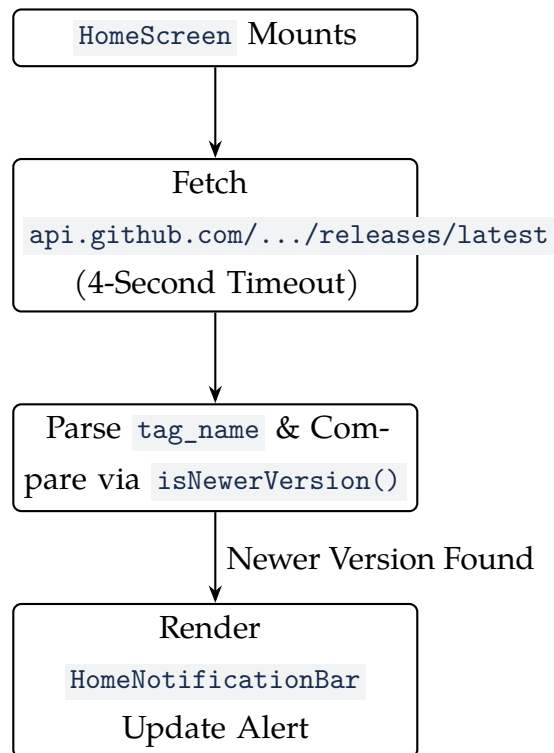
When a user navigates between channel tiles on the home screen:

1. Focused channel tile scales up smoothly by `1.05x` via React Native `Animated` or scale transform props.
2. Displays a high-contrast `ffffff` **3px border** with a 12px border radius.
3. Non-focused channel tiles remain at `opacity: 0.8` to emphasize spatial visual hierarchy across the 10-foot viewing distance.

6. IN-APP OTA UPDATER ARCHITECTURE

6.1 GitHub Release Check Lifecycle

In `src/Utils/updateChecker.ts`, the application silently checks for new production releases on startup without blocking the user:



6.2 Version Comparison Logic (`isNewerVersion`)

The version comparator parses semantic version strings (e.g. `v0.0.5` vs `v0.0.3`) into numeric array components:

```
// Semantic Version Comparer Algorithm
function isNewerVersion(latest: string, current: string): boolean {
  if (!latest || !current) return false;
  const lParts = latest.split('.').map((p) => parseInt(p, 10) || 0);
  const cParts = current.split('.').map((p) => parseInt(p, 10) || 0);

  for (let i = 0; i < Math.max(lParts.length, cParts.length); i++) {
    const l = lParts[i] || 0;
    const c = cParts[i] || 0;
    if (l > c) return true;
    if (l < c) return false;
  }
  return false;
}
\endcodeblock

% =====
% CHAPTER 7: LOW-LEVEL DESIGN (LLD) & DESIGN PATTERNS
% =====
\chapter{LOW-LEVEL DESIGN (LLD) \& DESIGN PATTERNS}

\section{Software Design Pattern Map}

\begin{longtable}{|L{1.5in}|L{2.0in}|L{2.6in}|}
  \hline
  \textbf{Design Pattern} & \textbf{Implementation File} & \textbf{System Purpose} \\ \hline
  \endhead
  \hline
  \endfoot
  Strategy Pattern & \code{PlayerScreen.tsx} & Dynamically selects player renderer (HLS vs Embed vs YouTube)
  \\\hline
  Proxy / Adapter Pattern & \code{ConsentSafeYouTubePlayer.tsx} & Wraps YouTube iframe with cookies \& \code{forcePlay()} handles.
  \\\hline
  Observer Pattern & \code{useIdleTimer.ts} & Listens to D-Pad activity and auto-hides player controls after 4s.
  \\\hline
  Flyweight / Lookup & \code{logoHelper.ts} & Pre-cached map of static WebP channel logo image requires.
  \\\hline
\end{longtable}

\section{Complete Module Hierarchy Map}

\begin{codeblock}
src/
|-- components/
|   |-- ConsentSafeYouTubePlayer.tsx (Proxy player for YouTube embeds)
|   |-- ChannelTile.tsx (Focused channel tile node)
|   |-- PlayerOverlay.tsx (Translucent D-Pad UI overlay)
|   |-- HomeNotificationBar.tsx (Breaking news / update status bar)
|   |-- CountryTabs.tsx (Country filter chip row)
|   |-- LanguageTabs.tsx (Language filter chip row)
|   +-- NoInternetScreen.tsx (Full-screen network outage screen)
|-- data/
|   +-- countries/india.json (Channel endpoint definitions)
|-- hooks/
|   |-- useLiveChannelResolver.ts (YouTube live stream ID resolver)
|   |-- useIdleTimer.ts (Overlay auto-hide timer hook)
|   +-- useNetworkStatus.ts (NetInfo & Google 204 connectivity hook)
|-- screens/
|   |-- HomeScreen.tsx (Grid channel catalog)
|   +-- PlayerScreen.tsx (Full-screen video player & zapping)
+-- utils/
    |-- playerScripts.ts (Extracted DOM autoplay & quality JS)

```